

CS-602 MJ SYSTEMS – II

Instruction Format Specification for a Two-Pass Assembler

Target Architecture: 32-bit x86 (IA-32), excluding instruction prefixes

Abstract

This assignment specifies how a two-pass assembler can parse selected IA-32 assembly instructions and translate them into variable-length machine-code bytes. It describes the instruction-byte layout, primary and 0F-extended opcodes, ModR/M and SIB addressing, immediate and displacement fields, relative branches, little-endian representation, and the DB/DW/DD and SECTION directives. The specification is intended as a compact implementation-oriented rule set rather than a complete definition of every IA-32 instruction.

1. Objectives and Scope

The assembler accepts 32-bit x86 assembly language and emits machine-code bytes and data bytes. Instruction prefixes are outside the scope. The specification concentrates on the encoding components required for this assignment: opcode, ModR/M, SIB, displacement, immediate, and data directives.

Component	Purpose
Opcode	Identifies the operation and, for some encodings, operand direction/size behavior.
ModR/M	Selects register operands and the main memory/register addressing form.
SIB	Encodes base + index $\times$ scale addressing when required.
Displacement	Adds a signed/unsigned address offset such as disp8 or disp32.
Immediate	Constant value embedded in the instruction.

2. Two-Pass Assembler

2.1 Pass 1

Pass 1 scans the source program, identifies labels and sections, determines instruction/data sizes, and builds a symbol table containing each label and its address. Forward references can therefore be recorded even before their final displacement is known.

PASS 1:

Read line -> identify label/opcode/operands -> determine size -> update location counter
-> enter labels in symbol table

2.2 Pass 2

Pass 2 scans the source again. The assembler now has symbol addresses, selects the final encoding, calculates displacements/relative offsets, writes immediate and displacement values in little-endian order, and emits the final bytes.

PASS 2:

Read line -> select opcode -> build ModR/M/SIB -> resolve symbols -> calculate offsets
-> encode fields -> emit machine code

3. IA-32 Instruction Byte Format

[Opcode][ModR/M][SIB][Displacement][Immediate]
 optional optional optional optional

The fields are variable-length and optional. A simple register instruction may need only an opcode and ModR/M byte, while a complex memory instruction may additionally require SIB and displacement. An immediate appears after the addressing fields when the instruction encoding requires one.

Field	Typical size	Meaning
Opcode	1 or more bytes	Operation code; 0F may introduce a two-byte opcode form.
ModR/M	1 byte	MOD, REG, and R/M fields.
SIB	1 byte	SCALE, INDEX, and

		BASE fields.
Displacement	1, 2, or 4 bytes depending on encoding	Address offset.
Immediate	1, 2, or 4 bytes depending on encoding	Constant operand.

4. Opcode Structure

4.1 Primary one-byte opcode

Many IA-32 instructions use a primary one-byte opcode. Some opcode families use bits within the opcode for operand direction or immediate-size behavior; these are described below. The exact meaning of an opcode must be taken from the instruction's encoding definition.

Example family:

ADD r/m32, r32 -> 01 /r

MOV r/m32, r32 -> 89 /r

4.2 0F two-byte opcode

For the two-byte opcode form, 0F is the escape/first opcode byte and a second byte selects the operation. For example, JNE uses the opcode bytes 0F 85 followed by a rel32 displacement.

Two-byte opcode form:

[0F][opcode2] [ModR/M ...]

Example:

JNE rel32 -> 0F 85 cd

4.3 Direction bit (d)

In opcode families that define a d bit, d selects the direction of the register and r/m operands. For the common register/memory forms, d=0 and d=1 correspond to opposite operand directions as defined by that instruction's encoding. The assembler must use the specific instruction table rather than assume that every opcode contains a d bit.

4.4 Sign-extension bit (s)

In opcode families that define an s bit, s indicates that a smaller immediate is sign-extended to the operation's operand size. A common IA-32 example is an instruction

with an imm8 form that is sign-extended to 32 bits. Again, s is meaningful only in opcode families that define it.

5. Register Encoding Lookup Table

Binary	32-bit register
000	EAX
001	ECX
010	EDX
011	EBX
100	ESP
101	EBP
110	ESI
111	EDI

6. ModR/M Byte

Bit: 7 6 | 5 4 3 | 2 1 0
MOD | REG | R/M

MOD	Meaning
00	Memory addressing; normally no displacement, with special cases.
01	Memory addressing with disp8.
10	Memory addressing with disp32.
11	Register-direct operand; R/M selects a register.

6.1 REG field

The REG field is three bits. For instructions using /r, it normally selects a register operand. For opcode-extension encodings, the REG field may instead select an operation subfunction (/0 through /7).

6.2 R/M field and important 32-bit cases

MOD	R/M	Meaning
11	000–111	Register selected by the 3-

		bit register table.
00	000–011,110,111	Memory using EAX, ECX, EDX, EBX, ESI, or EDI as base.
00	100	SIB follows.
00	101	disp32-only memory addressing (no EBP base).
01/10	100	SIB follows, with disp8/disp32 respectively.
01/10	others	Base register selected by R/M plus disp8/disp32.

7. SIB Byte

Bit: 7 6 | 5 4 3 | 2 1 0
SCALE | INDEX | BASE

SCALE	Multiplier
00	×1
01	×2
10	×4
11	×8
INDEX	Register / special meaning
000	EAX
001	ECX
010	EDX
011	EBX
100	No index (special SIB meaning)
101	EBP
110	ESI
111	EDI
BASE	Register / special meaning
000	EAX
001	ECX
010	EDX

011	EBX
100	ESP
101	EBP (or disp32-only when MOD=00)
110	ESI
111	EDI

7.1 Required complex addressing example

For:

[EBX + ESI*4 + 10h]

Base = EBX (011), Index = ESI (110), Scale = 4 (10), and displacement = 10h.

Therefore the SIB bits are 10 110 011 = B3h. With MOD=01 for disp8 and R/M=100 to indicate SIB, the ModR/M byte is 01 000 100 = 44h when the REG field is EAX (000). The complete instruction's opcode and REG field depend on the instruction being encoded.

8. Addressing Modes

Mode	Example	Encoding idea
Register	EAX	MOD=11; R/M selects register.
Register indirect	[EBX]	Memory with EBX as base.
Base + disp8	[EBX+10h]	MOD=01 plus 1-byte displacement.
Base + disp32	[EBX+12345678h]	MOD=10 plus 4-byte displacement.
Base + index	[EBX+ESI]	SIB with scale $\times 1$.
Base + index $\times$ scale	[EBX+ESI*4]	SIB with scale $\times 4$.
Base + index $\times$ scale + displacement	[EBX+ESI*4+10h]	SIB + displacement.

9. Immediate and Displacement Encoding

Field	Size	Use
imm8	1 byte	8-bit immediate when the

		instruction encoding permits it.
imm16	2 bytes	16-bit immediate in 16-bit operand-size forms.
imm32	4 bytes	32-bit immediate in 32-bit operand-size forms.
disp8	1 byte	Short displacement; sign-extended for address calculation.
disp32	4 bytes	32-bit displacement.

Immediate and displacement fields are placed after the opcode/addressing fields in the order specified by the instruction encoding. Multi-byte numeric values are stored little-endian.

0x12345678 -> 78 56 34 12

0x1234 -> 34 12

0x10 -> 10

10. Relative JMP and JNE

Relative branch instructions encode a displacement from the address immediately following the branch instruction. The basic calculation is:

relative displacement = target address - address of next instruction

10.1 JMP rel32 example

Assume:

JMP at 1000h

JMP instruction size = 5 bytes

TARGET = 1025h

Next instruction = 1000h + 5 = 1005h

Displacement = 1025h - 1005h = 0020h

Little-endian = 20 00 00 00

JMP rel32 = E9 20 00 00 00

10.2 JNE rel32 example

Assume:

JNE at 2000h

JNE rel32 size = 6 bytes

TARGET = 1FF0h

Next instruction = 2000h + 6 = 2006h

Displacement = 1FF0h - 2006h = -16h

32-bit value = FFFFFFFEAh

Little-endian = EA FF FF FF

JNE rel32 = 0F 85 EA FF FF FF

For a short conditional jump, an instruction-specific short opcode is followed by a signed rel8 displacement. The assembler chooses the form according to the supported syntax/encoding and the final displacement range.

11. Assembler Directives

Directive	Allocation	Example	Bytes
DB	1 byte	DB 25h	25
DW	2 bytes	DW 1234h	34 12
DD	4 bytes	DD 12345678h	78 56 34 12

11.1 SECTION .text

SECTION .text begins the code section. Instructions are placed here and the location counter advances by the encoded length of each instruction.

11.2 SECTION .data

SECTION .data begins the initialized-data section. DB/DW/DD reserve and initialize the specified number of bytes. The location counter advances by the number of bytes emitted.

SECTION .data

num1 DB 25h

num2 DW 1234h

num3 DD 12345678h

12. Complete Sample Encoding Walkthrough

The following examples demonstrate the exact method used by the assembler.

12.1 MOV EAX, EBX

Instruction: MOV EAX, EBX

Encoding form: MOV r32, r/m32 -> 8B /r

Opcode = 8B

MOD = 11 (register)

REG = EAX = 000

R/M = EBX = 011

ModR/M = 11 000 011 = C3

Final machine code: 8B C3

12.2 ADD EAX, EBX

Instruction: ADD EAX, EBX

Encoding form: ADD r/m32, r32 -> 01 /r

Opcode = 01

MOD = 11

REG = EBX = 011

R/M = EAX = 000

ModR/M = 11 011 000 = D8

Final machine code: 01 D8

12.3 MOV ECX, [EBX + ESI*4 + 10h]

Instruction: MOV ECX, [EBX + ESI*4 + 10h]

Encoding form: MOV r32, r/m32 -> 8B /r

Opcode = 8B

MOD = 01 (disp8)

REG = ECX = 001

R/M = 100 (SIB follows)

ModR/M = 01 001 100 = 4C

SCALE = 4 = 10

INDEX = ESI = 110

BASE = EBX = 011

SIB = 10 110 011 = B3

Displacement = 10h -> 10

Final machine code: 8B 4C B3 10

This example demonstrates all major addressing fields: opcode, ModR/M, SIB, and an 8-bit displacement.

13. Example Symbol Resolution in Two Passes

```
SECTION .text
```

```
START:
```

```
    MOV EAX, 1
```

```
    JMP DONE
```

```
    MOV EBX, 2
```

```
DONE:
```

```
    ADD EAX, EBX
```

In Pass 1, START and DONE are assigned addresses after instruction sizes are determined. In Pass 2, the JMP target DONE is known, so the assembler calculates its relative displacement from the address of the next instruction and emits the correct bytes.

14. Encoding Algorithm Summary

1. Parse the mnemonic and operands.
2. Determine operand sizes and addressing mode.
3. Select the exact instruction encoding and opcode.
4. Determine whether ModR/M is required.
5. If memory addressing requires it, construct the SIB byte.
6. Determine disp8/disp32 and/or immediate size.
7. Resolve symbols and calculate relative branch offsets.
8. Encode multi-byte numeric fields in little-endian order.
9. Concatenate fields in the specified instruction order.
10. Emit the final machine-code bytes.

15. Conclusion

This specification provides a practical encoding model for a two-pass IA-32 assembler. By separating opcode selection, ModR/M and SIB construction, displacement/immediate encoding, symbol resolution, and little-endian byte emission,

the assembler can translate variable-length 32-bit x86 instructions into deterministic machine-code bytes. The same process extends to additional IA-32 instructions by adding their official opcode and operand-encoding definitions.

References

1. Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference.
2. Intel 64 and IA-32 Architectures Software Developer's Manual, instruction format and addressing-mode tables.
3. Course/assignment specification supplied by the instructor.

Team Members

Name	Roll No	GitHub links
Sarthak Ubale	25111069	https://github.com/sarthak-026/Sys-2
Akash Gaikar	25112035	https://github.com/Akash-Gaikar/CS-602-MJ-SYSTEM---II.git
Amol Barde	25111007	https://github.com/Amolbarde/CS_602_System-2.git
Vikas Gawari	25111018	https://github.com/vikasgawari1929/CS_602_System-2.git
Akshay Petkar	25111045	https://github.com/akshaypetkar2003-boop/CS-602-MJ-System-II